

Multi
–level Linked List
–Modified Deletion

1)Delete At Beginning

Program:

```
void deleteAtBeginning()
{
    if (head == nullptr)
    {
        cout << "List is empty." << endl;
        return;
    }

    Node *temp = head;
    head = head->next;

    // Recursively free the entire child branch attached to
    this node
    destroyNodeRecursively(temp->child);

    free(temp);
    cout << "Deleted from beginning." << endl;
}
```

Program Analysis – Modification

```
destroyNodeRecursively(temp->child);
```

After, deleting the node at beginning, we delete children nodes linked to it.

Time complexity

Best case

Deletion of node at beginning, like earlier remains the same: $O(1)$.

The head node or head pointer points to first Node doesn't have any child, then time complexity remains the same: $O(1)$.

Worst Case

Deletion of node at beginning, like earlier remains the same: $O(1)$.

```
destroyNodeRecursively(temp->child);
```

This dominates the running time.

Suppose the deleted head has k descendants (all nodes reachable through child and their next links).

Since every descendant is visited exactly once,

Time Complexity = $O(k)$

where

- *k = number of nodes in the child subtree.*

<i>Cases</i>	<i>Time Complexity</i>
<i>Best Case</i>	$O(1)$
<i>Worst Case</i>	$O(k)$, $k = \text{number of nodes in the child subtree.}$
<i>Average Case = Worst Case</i> [<i>No true Average Case</i> here.]	$O(k)$

Space complexity

Parametric Input Space Complexity: It accepts absolutely zero parameters.

The parametric input space is completely constant, requiring 0 bytes of explicit parameter memory, making it $O(1)$.

Full Data Input (Holistic) Space Complexity:

Let N be the number of nodes in the main top – level linked list,

and let M be the total number of child nodes spread across all branches.

The total memory occupied by the holistic data structure is $O(N + M)$.

- 1) *No child node [Right Skewed] := Full Data Input : $O(N + 0) =: O(N)$*
- 2) *With Child node but with 1 or `c` next node [Left Skewed] := Full Data Input : $O(c + M) =: O(M)$ [where `c` is constant.]*

Auxiliary Space:

1) Best Case (No children): If $\text{temp} \rightarrow \text{child}$ is nullptr, the recursive function likely just hits its base case immediately and returns. Auxiliary Space = $O(1)$.

2) Worst Case (with children): Auxiliary Space for deleting next node remains, $O(1)$ with `M` number of child node : $O(1) + O(M) = O(M)$.

2)Delete At End

Program:

```
void deleteAtEnd()
{
    if (head == nullptr)
        return;

    // If there is only one element in the main list
    if (head->next == nullptr)
    {
        destroyNodeRecursively(head->child); // Free
children first
        free(head);
        head = nullptr;
        cout << "Deleted from end." << endl;
        return;
    }

    Node *temp = head;
    while (temp->next->next != nullptr)
    {
        temp = temp->next;
    }

    // temp->next is the last node. Free its children first.
    destroyNodeRecursively(temp->next->child);

    free(temp->next);
    temp->next = nullptr;
    cout << "Deleted from end." << endl;
}
```

Program Analysis – Modification

if there is only one node linked with head pointer then:

```
if (head->next == nullptr)
{
    destroyNodeRecursively(head->child); // Free
children first
    free(head);
    head = nullptr;
    cout << "Deleted from end." << endl;
    return;
}
```

It will not only free the head node or First node linked with Head Pointer, but also the child nodes associated with it.

If there is N number of node linked with Linked List, then:

```
Node *temp = head;
while (temp->next->next != nullptr)
{
    temp = temp->next;
}

// temp->next is the last node. Free its children first.
destroyNodeRecursively(temp->next->child);
```

It will locate the N^{th} node and destroy its associated children.

Time complexity

Lets assume there is N no. of next nodes and M no. of associated child nodes.

Best Case Time Complexity: $O(1)$

The best – case scenario occurs when the function can return almost immediately without having to traverse any lists. This happens in two situations:

- 1. The main list is empty ($head == nullptr$).
The function checks this condition and returns instantly.***
- 2. The main list has only 1 node, and it has 0 child nodes ($N = 1, M = 0$). The function enters the $if (head \rightarrow next == nullptr)$ block, calls the recursive destroy function on a $nullptr$ (which does nothing), frees the single node, and returns.***

In both of these scenarios, the time taken is constant: $O(1)$.

Worst Case Time Complexity: $O(N + M)$

The worst – case scenario occurs when the function has to do the maximum amount of traversal and deletion. This happens when:

- 1. The main list is very long. The while loop has to traverse through $(N - 1)$ nodes just to find the second – to – last node.
This traversal takes $O(N)$ time.***
- 2. The last node has a massive chain of child nodes. Once the last node is found, $destroyNodeRecursively()$ is called. If all M child nodes in your entire data structure happen to be attached to this specific last node, the recursive function must visit and free every single one of them. This takes $O(M)$ time.***

Combining the long main list traversal and the deep child list deletion gives a worst – case time complexity of $O(N + M)$.

Space Complexity

1. Input Space Complexity

Input space complexity refers to the memory required to store the data that the algorithm operates on. This can be split into two perspectives:

A) Parameter Input Space: $O(1)$

The function signature is void deleteAtEnd(). It takes exactly zero parameters. The head pointer is accessed as a global variable. Therefore, the memory explicitly allocated for passing arguments to this specific function is constant, or $O(1)$.

B) Full Data Input Space: $O(N + M)$

This refers to the memory occupied by the entire data structure the function is manipulating in memory. If the data structure contains N nodes in the main horizontal list and a total of M child nodes branching off of those main nodes, the total memory footprint of the structure is directly proportional to the total number of nodes.

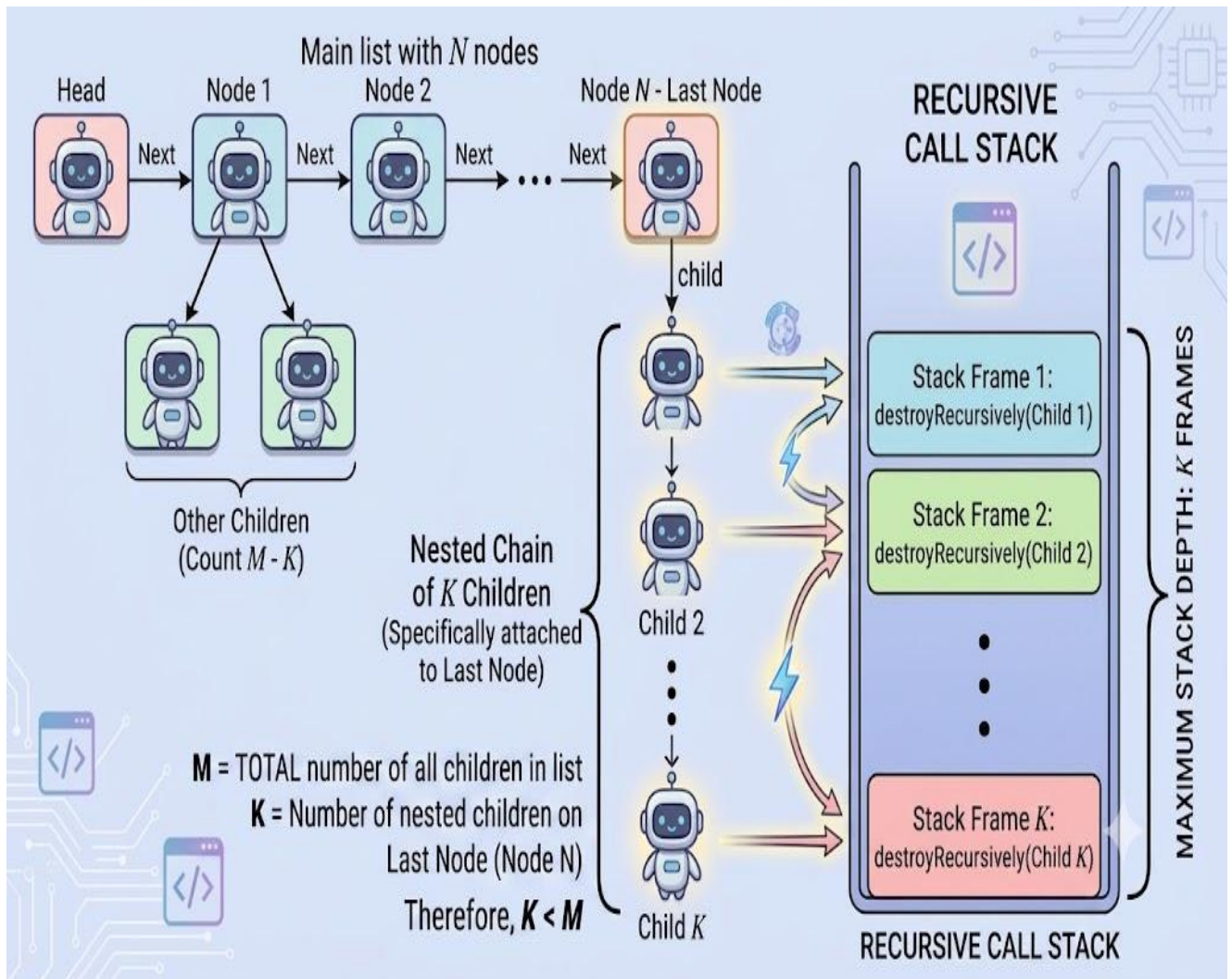
Therefore, the full data space is $O(N + M)$.

2. Auxiliary Space: $O(K)$ (where $K \leq M$)

Auxiliary space is the temporary, extra space used by the algorithm during its execution, excluding the input data itself.

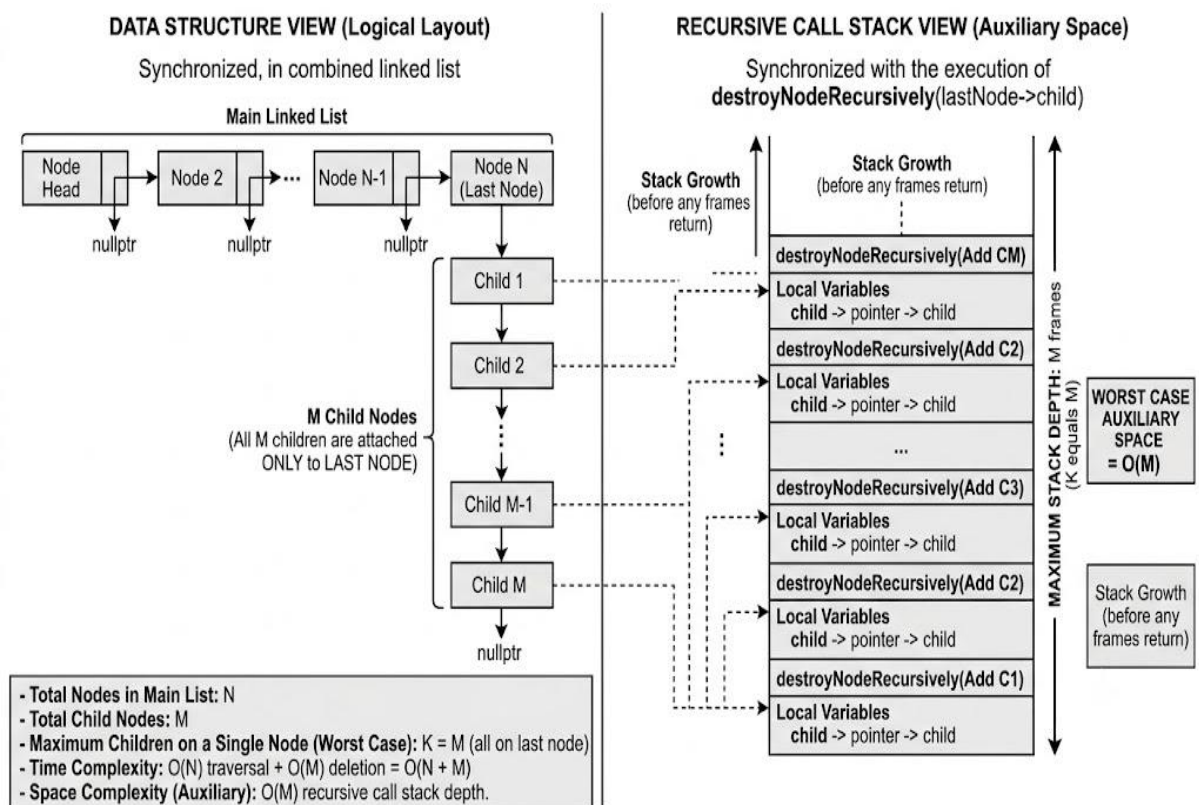
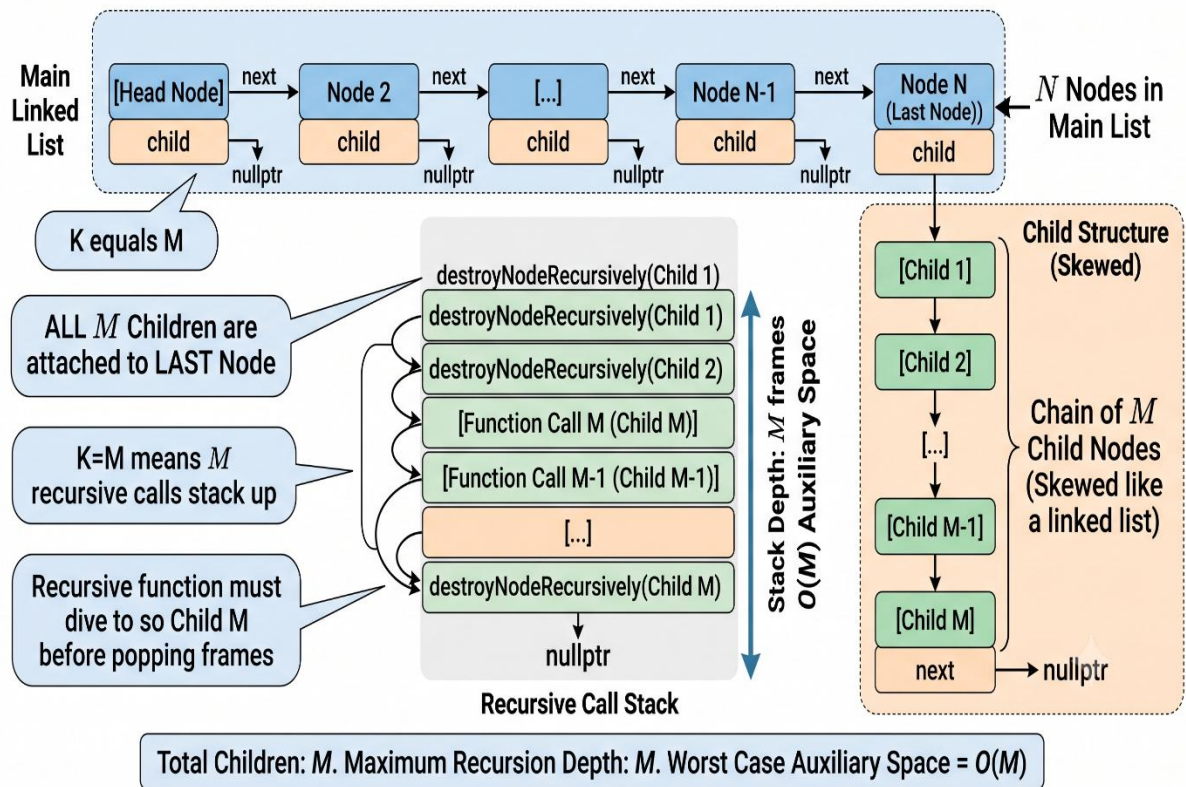
For this function, the auxiliary space is entirely dependent on the recursive call to `destroyNodeRecursively()`.

- ***Local Variables ($O(1)$): The function only creates one local variable, `Node * temp = head;`, which requires a constant amount of memory for a single pointer.***
- ***Call Stack from Recursion ($O(K)$): The function calls `destroyNodeRecursively()` to delete the children of the last node. Because this helper function is recursive, every step of its recursion adds a new frame to the call stack in memory.***
 - ***If K represents the number of child nodes attached specifically to the last node in the main list, the call stack will reach a maximum depth of K frames.***



→ ***In the absolute worst – case scenario, if every single child node in your entire structure is attached to this last node, K equals M , making the worst – case auxiliary space $O(M)$.***

Absolute WORST-CASE AUXILIARY SPACE ($O(M)$)



Perfectly Balanced Tree:

If the child nodes are balanced (each node has two children, filling out the levels evenly), the height of the tree is drastically reduced. The recursion only ever goes as deep as the height of the tree.

Auxiliary space : $O(\log_2 K)$, $K < M$

And if $K = M$ amount of child nodes attached to last node, then:

Auxiliary space : $O(\log_2 M)$, where $K = M$

Delete At Position

Program:

```
void deleteAtPosition()
{
    int pos;
    cout << "Enter position to delete: ";
    cin >> pos;

    if (head == nullptr)
        return;

    Node *temp = head;

    if (pos == 1)
    {
        head = head->next;
        destroyNodeRecursively(temp->child); // Free
children first
        free(temp);
        cout << "Deleted at position " << pos << endl;
        return;
    }

    for (int i = 1; i < pos - 1 && temp != nullptr; i++)
    {
        temp = temp->next;
    }

    if (temp == nullptr || temp->next == nullptr)
    {
        cout << "Position out of range." << endl;
        return;
    }

    Node *toDelete = temp->next;
    temp->next = toDelete->next;
```

```

// Free children attached to the node being deleted
destroyNodeRecursively(toDelete->child);

free(toDelete);
cout << "Deleted at position " << pos << endl;
}

```

Program Analysis – Modification

if position = 1

```

if (pos == 1)
{
    head = head->next;
    destroyNodeRecursively(temp->child); // Free
children first
    free(temp);
    cout << "Deleted at position " << pos << endl;
    return;
}

```

*It free the head node or First Node pointed by head pointer ,
then it can destroy its child through:*

```

    destroyNodeRecursively(temp->child); // Free
children first

```

if position = pos

```
Node *toDelete = temp->next;
temp->next = toDelete->next;

// Free children attached to the node being deleted
destroyNodeRecursively(toDelete->child);

free(toDelete);
```

When it is directed to delete a specific position `pos`, then :

```
destroyNodeRecursively(toDelete->child);
```

the function destroy the specific node's child.

Time Complexity – Analysis

To ensure the analysis is precise, let's define the variables involved:

- *N: The total number of nodes in the main horizontal linked list.*
- *K: The number of child nodes attached specifically to the node being deleted.*
- *M: The total number of child nodes across the entire data structure.*

1. Best Case Time Complexity: $O(1)$

The best – case scenario happens when the function can execute and return with the absolute minimum number of operations. This occurs under two specific conditions:

1. The list is completely empty ($head == nullptr$).

The function checks this and returns immediately.

2. Deleting the first node, and it has no children

($pos = 1$ and $K = 0$). The function enters the

if ($pos == 1$) block, skips the for loop traversal,

calls the recursive function on $nullptr$ (which does nothing), frees the node, and returns.

Because no loops are traversed and no recursion occurs, the time taken is constant: $O(1)$.

2. Worst Case Time Complexity: $O(N + M)$

The worst – case scenario happens when the function is forced to do the maximum amount of work for both traversal and deletion. This occurs when:

1. Maximum Traversal: Attempting to delete the very last node in the list ($pos = N$), or you enter a position that is out of bounds ($pos > N$). The for loop must run $N - 1$ times to reach the end of the list. This takes $O(N)$ time.

2. Maximum Deletion: The node of which deletion happens to hold every single child node in the entire data structure ($K = M$). The `destroyNodeRecursively()` function must traverse and free all M nodes. This takes $O(M)$ time.

Combining the full list traversal and the full child structure deletion gives the absolute worst – case time of $O(N + M)$.

Average Case[$O(N + K)$]

Deleted position from 2 to N , for loop runs from 1 to $N - 2$.

And when the position is uniformly distributed from 1 to N :

(Average Case i. e. Undetermined Position)

When we want to delete 1st node , loop runs upto 0 time.

When we want to delete 2nd node , loop runs upto 0 time.

When we want to delete 3rd node , loop runs upto 1 time.

.

.

When `n - 1` node is to be deleted :

Upper Bound : pos - 2, when pos = n - 1, then :

loop runs: (n - 1 - 2) = n - 3 times.

When `n` node is to be deleted :

Upper Bound : pos - 2, when pos = n, then :

loop runs: (n - 2) = n - 2 times.

When we want to delete $\sum_{i=1}^n i$ nodes , loop runs $\sum_{i=1}^{n-2} i$

Now, as we take all the possibilites then , we take the best case also i. e.

O(1), when position = 1 . Considering `T(P)` is time complexity of position `P` .

$$T(P) = \begin{cases} 1 & , \text{when } P = 1 . \\ 1 & , \text{when } P = 2 . \\ P - 2 & , \text{when } P = 3, 4, \dots, N. \end{cases}$$

Now, If there is `N` no. of nodes there we can delete data at N positions.

For position 1, the probability of deletion is $\frac{1}{N}$.

For position 2, the probability of deletion is $\frac{1}{N}$.

.

.

For position N, the probability of deletion is $\frac{1}{N}$.

Hence probability of deletion will always remain $\frac{1}{N}$.

$\therefore$ Average Time Complexity is : probability of deletion $\times$ (number of possible iterations + constant time complexities for left over positions) .

$$= \left(\frac{1}{N} \times 1 + \frac{1}{N} \times 2 + \dots + \frac{1}{N} \times N - 2 \right) + \frac{1}{N} \times 1 + \frac{1}{N} \times 1$$

$$= \frac{1}{N} \times \{(1 + 2 + 3 + \dots + N - 2) + 1 + 1\}$$

$$= \frac{1}{N} \times \left\{ \sum_{i=1}^{n-2} i + 2 \right\}$$

$$= \frac{1}{N} \left\{ \frac{(n-2)(n-1)}{2} + 2 \right\}$$

$$= \frac{1}{N} \left\{ \frac{n^2 - 3n + 6}{2} + 2 \right\}$$

$$= \left\{ \frac{n^2 - 3n + 6}{2n} + \frac{2}{n} \right\}$$

$$= \left\{ \frac{n^2}{2n} - \frac{3n}{2n} + \frac{6}{2n} + \frac{2}{n} \right\}$$

$$= \left\{ \frac{n}{2} - \frac{3}{2} + \frac{3}{n} + \frac{2}{n} \right\}$$

Appplying Big Theta in the equation:

$$= \Theta\left(\frac{n}{2}\right) - \Theta\left(\frac{3}{2}\right) + \Theta\left(\frac{3}{n}\right) + \Theta\left(\frac{2}{n}\right)$$

$$= \frac{1}{2} \times \Theta(n) - \frac{3}{2} \times \Theta(1) + 3 \times \Theta\left(\frac{1}{n}\right) + 2 \times \Theta\left(\frac{1}{n}\right)$$

Removing all constants:

$$= \Theta(n) + \Theta\left(\frac{1}{n}\right) + \Theta\left(\frac{1}{n}\right)$$

When combining two Big Theta Θ terms , largest growth rate is the result:

$\rightarrow \Theta(N)$ grows linearly with `N` .

$\rightarrow \Theta\left(\frac{1}{N}\right)$ decreases to a constant , i. e. when N grows larger , $\frac{1}{N}$ approaches to 0.

This we can prove by limit and continuity also:

$$\lim_{n \rightarrow \infty} \left(\frac{1}{n}\right) \Rightarrow \frac{1}{\infty} = 0 \left[\frac{c}{\infty} = 0 , \text{ where } c \text{ is constant} \right].$$

As $\Theta(N)$'s growth rate is higher , therefore , average case is $\Theta(N)$.

Hence, Average Time Complexity is : $\Theta(N)$.

Note , by relation: $\Omega \leq \Theta \leq O$, we get:

$$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Rightarrow c_1 \times 1 \leq \frac{n^2 - 3n + 6}{2n} + \frac{2}{n} \leq c_2 \times N$$

Average Case Time Complexity[Most Simplest Alternative Apporach]:
(Simplest approach)

For position 1, the probability of deletion is $\frac{1}{N}$.

For position 2, the probability of deletion is $\frac{1}{N}$.

.

.

For position N, the probability of deletion is $\frac{1}{N}$.

Hence probability of deletion will always remain $\frac{1}{N}$, for position 1 to N.

Average case complexity : Probability of deletion $\times$ $\left(\begin{array}{c} \text{Summation} \\ \text{of number of} \\ \text{Position.} \end{array} \right)$

$$= \frac{1}{N} \times \left(\sum_{i=1}^N i \right)$$

$$= \frac{1}{N} \times \left(\frac{N(N+1)}{2} \right)$$

$$= \frac{N+1}{2}$$

$$\text{or, } \frac{N}{2} + \frac{1}{2}$$

Appplying Big Theta in the equation:

$$= \Theta\left(\frac{N}{2}\right) + \Theta\left(\frac{1}{2}\right)$$

Now removing the constant $\frac{1}{2}$, we get:

$$= \Theta(N) + \Theta(1)$$

$$= \Theta(N).$$

Hence, Average Time Complexity is : $\Theta\left(\frac{N+1}{2}\right)$ or $\Theta(N)$.

Now,

Note , by relation: $\Omega \leq \Theta \leq O$, we get:

$$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Rightarrow c_1 \times 1 \leq \frac{N+1}{2} \leq c_2 \times N$$

Now child node deletion

Deletion: Deleting the specific child nodes attached to this middle node takes $O(K)$ time.

Adding the traversal and deletion times together yields an average complexity of $O(N + K)$.

Space Complexity

Let's keep our variables consistent:

- *N: Total nodes in the main list.*
- *M: Total child nodes in the entire data structure.*
- *K: The number of child nodes attached specifically to the node being deleted ($K \leq M$).*

1. Input Space Complexity

- *Parameter Input Space: $O(1)$*

The function signature is `void deleteAtPosition()`. It takes zero parameters. Therefore, the memory explicitly allocated for passing arguments is constant, $O(1)$.

- *Full Data Input Space: $O(N + M)$*

The total memory footprint required to hold entire data structure in RAM is linearly proportional to the total number of nodes (main nodes + child nodes), which is $O(N + M)$.

2. Auxiliary Space Complexity

Auxiliary space is the temporary extra memory your function uses while running. For `deleteAtPosition()`, this comes from two places: Local Variables and the Recursive Call Stack.

A. Local Variables: $O(1)$

Inside the function, you allocate a few basic variables:

*int pos; Node * temp;*

int i; (inside the loop)

*Node * toDelete;*

These take up a fixed, tiny amount of memory regardless of how big the linked list is. This part is strictly $O(1)$.

B. Recursive Call Stack:

The true auxiliary space depends entirely on the call to `destroyNodeRecursively()`.

Just like your analysis of the time complexity, the space complexity hinges on the shape and size of the "K" child nodes attached to the deleted node.

Here is how the Auxiliary Space breaks down by scenario:

Scenario	Condition	Auxiliary Space
Best Case	<i>The node being deleted has 0 child nodes ($K = 0$). The recursive function returns instantly without pushing frames to the stack.</i>	$O(1)$
Worst Case	<i>The node being deleted has K child nodes, from a specific node contains in the main list . The recursion dives K levels deep. ($K \leq M$) (where M is total child nodes existed in the list)</i>	$O(K)$
Balanced Case	<i>The node being deleted has K child nodes, and they form a Perfectly Balanced Tree. The recursion only dives to the height of the tree. ($K \leq M$) (where M is total child nodes existed in the list)</i>	$O(\log_2 K)$
Absolute Global Worst Case	<i>The node being deleted holds every single child node in the entire structure ($K = M$), and they form a straight line (Skewed).</i>	$O(M)$
